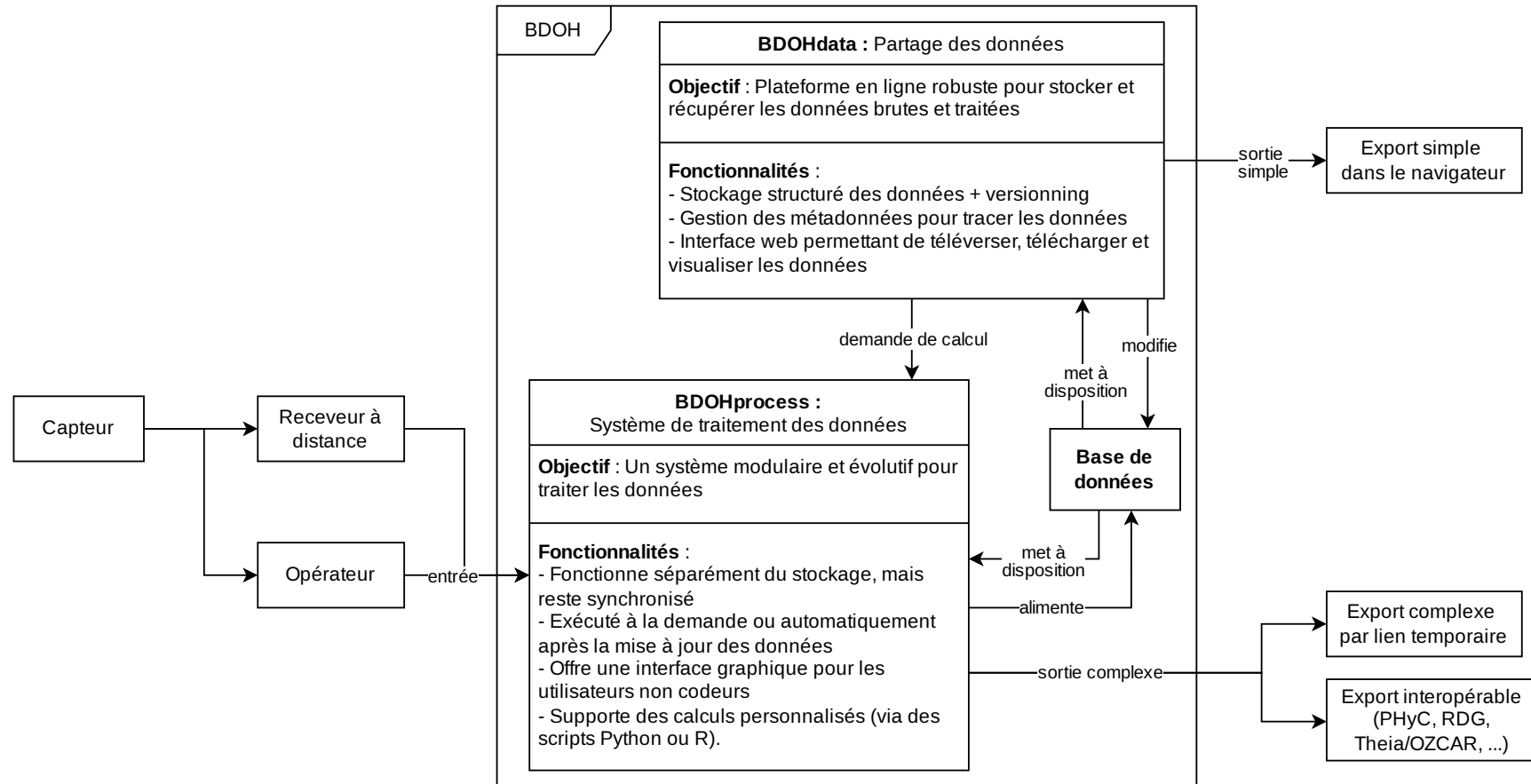


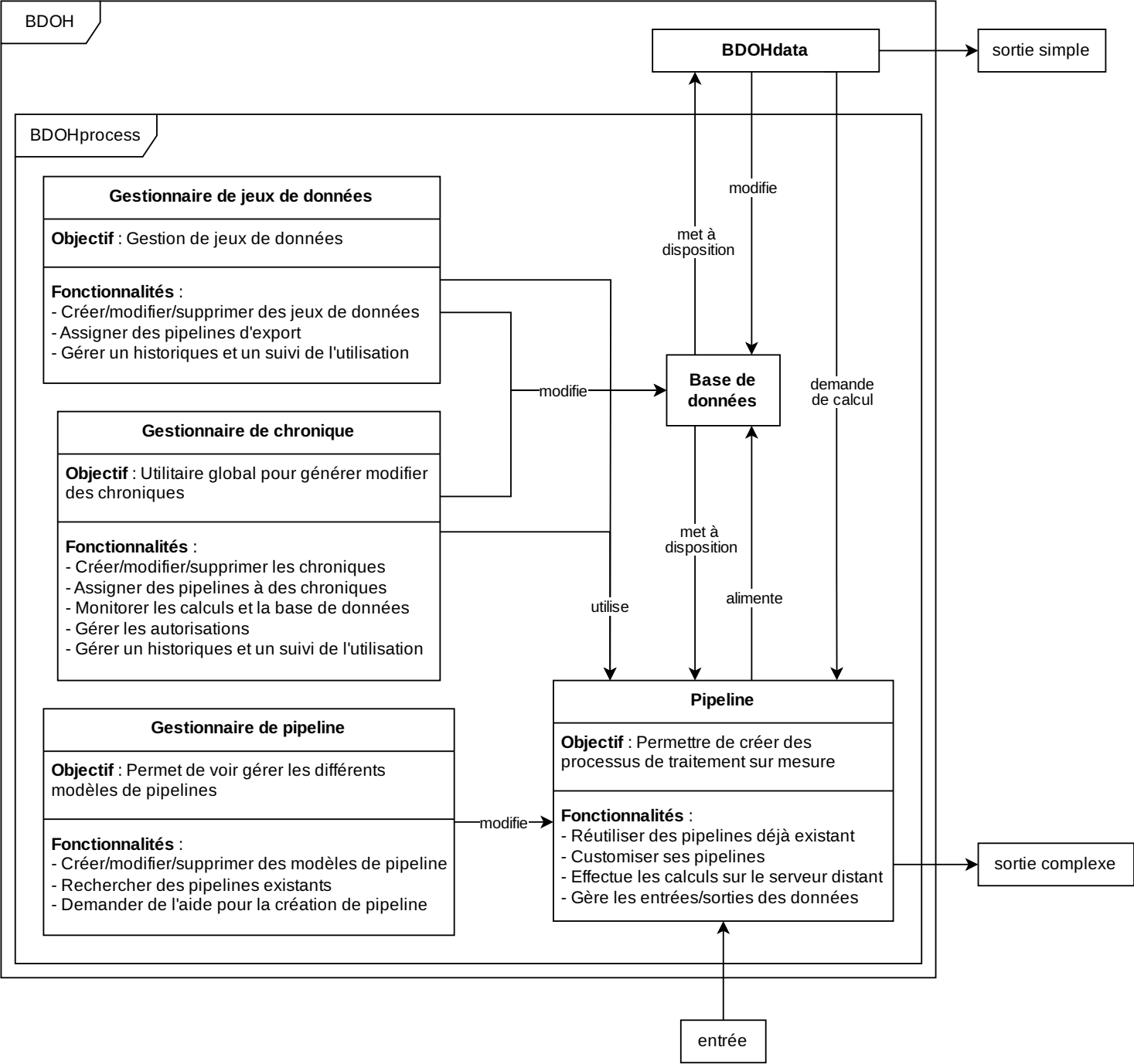
BDOH

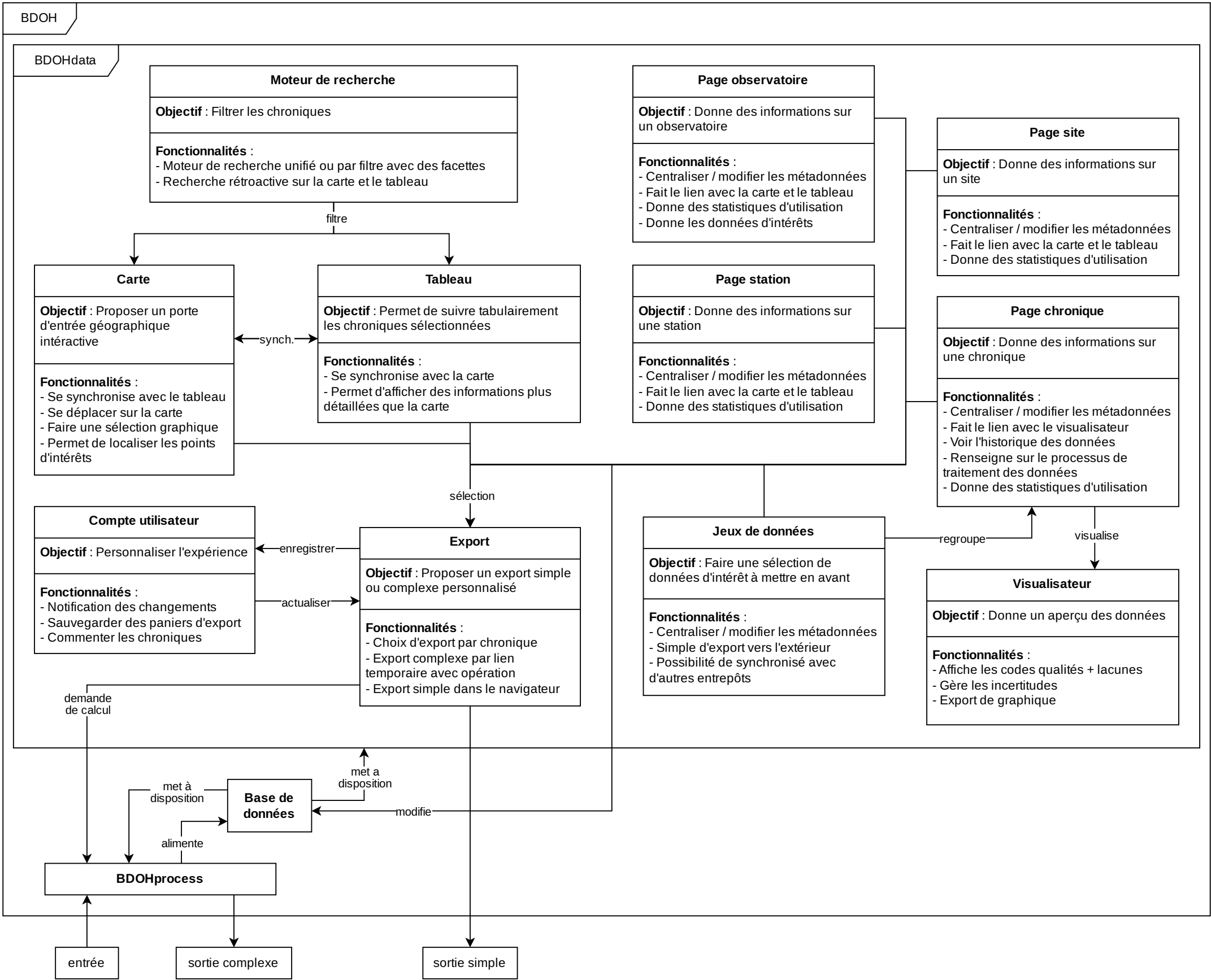
Architecture



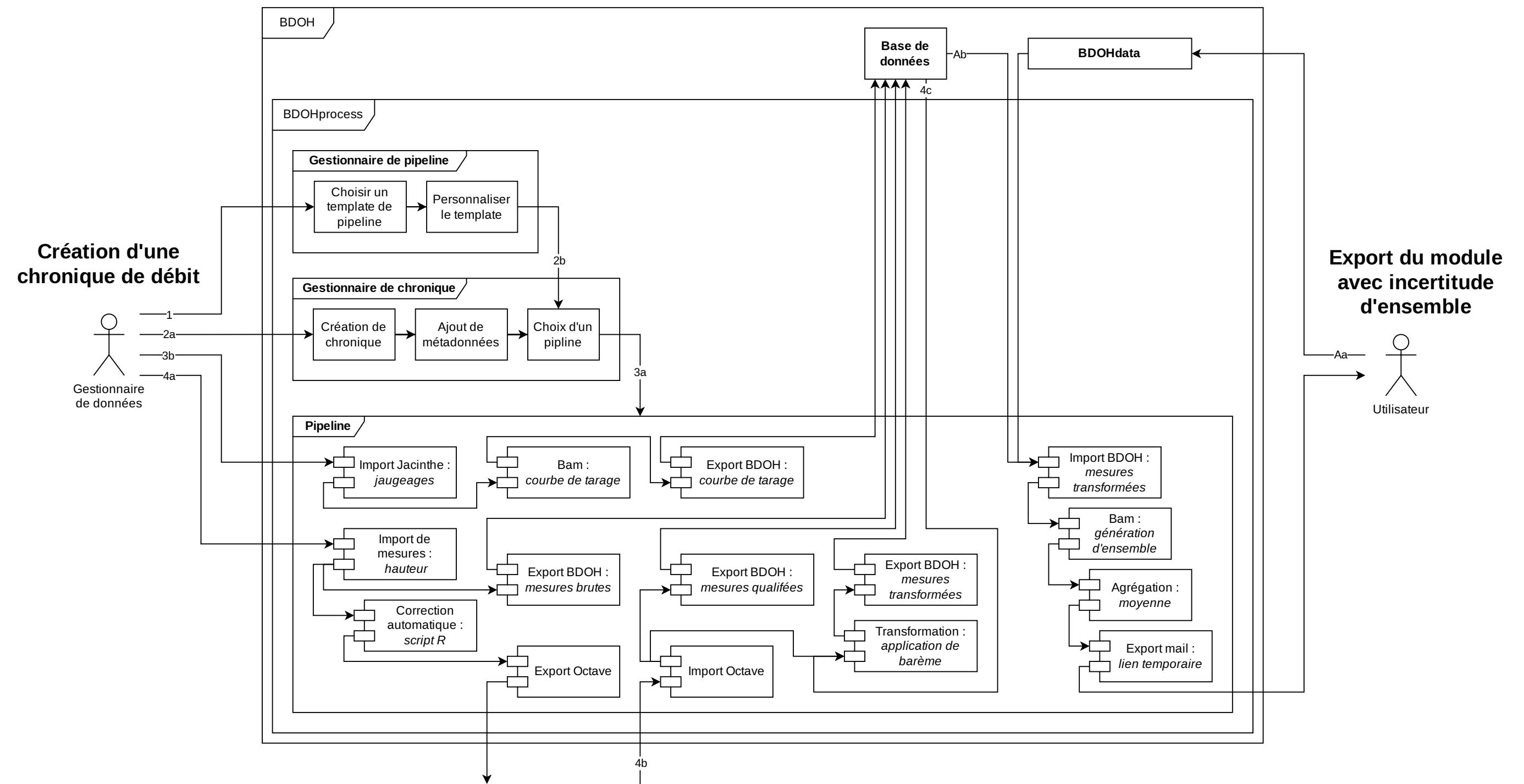
BDOHprocess

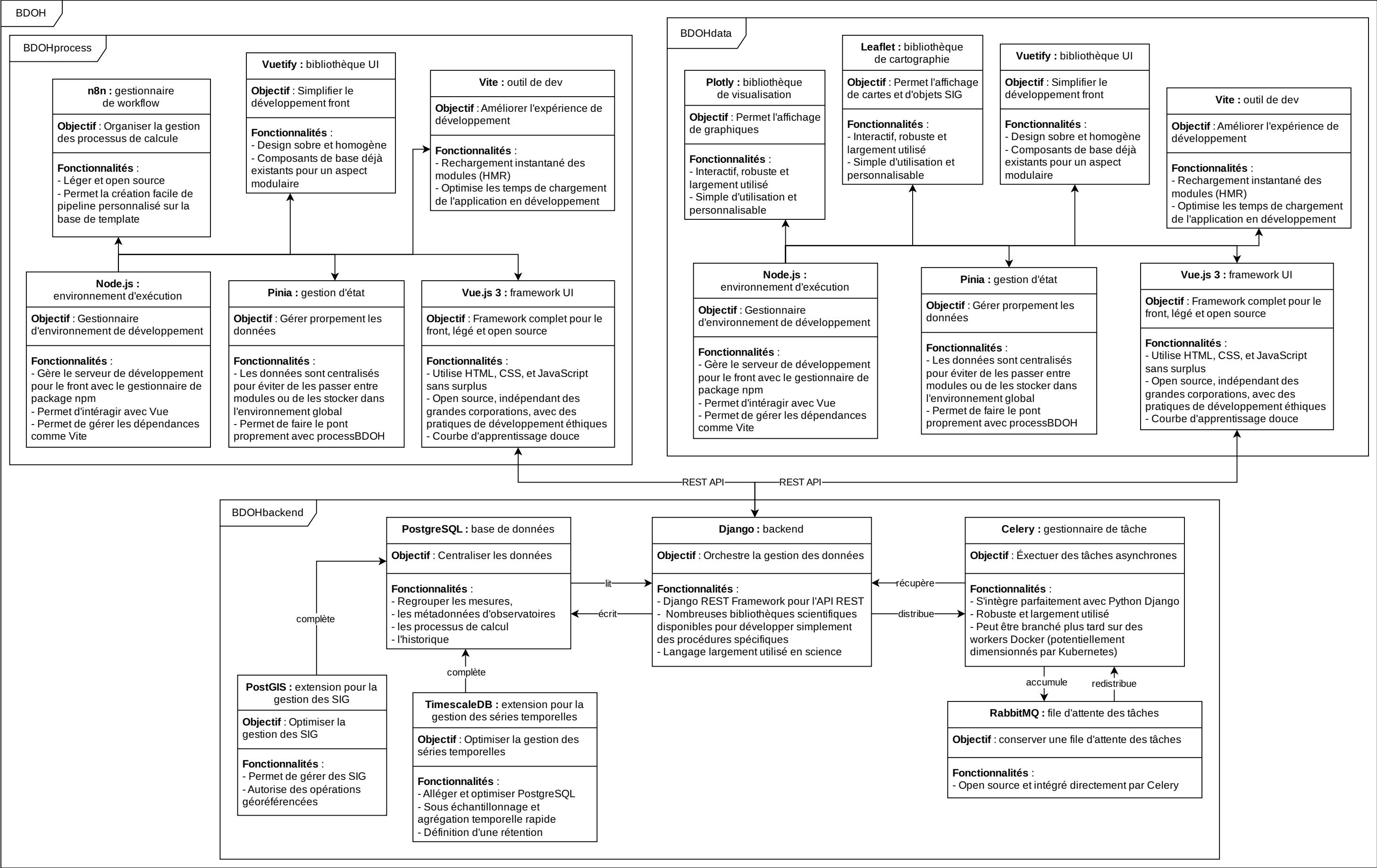
Architecture





BDOHprocess
Cas d'utilisations





BDOH

Doutes sur les technologies

Doutes : Frontend

- **Pinia** : Pas strictement nécessaire, sauf si une gestion d'état complexe. Sinon utiliser la fonction « **provider/inject** » intégrée de **Vue** pour partager l'état dans les cas à petite échelle.
- **Mapbox** ou **Deck.gl** si besoin d'une meilleure visualisation.
- **Plotly** peut être lourd : Envisager des alternatives basées sur **WebGL** comme **ECharts** pour un rendu plus fluide.
- Interface utilisateur de recherche et de filtrage : l'indexation est nécessaire pour des recherches rapides. Pensez à utiliser **Elasticsearch** ou l'index **PG_trgm** pour **PostgreSQL**.
- Le contrôle des versions des flux de travail sera essentiel.
- L'interface utilisateur doit gérer les commentaires sur les tâches asynchrones (par exemple, un tableau de bord de file d'attente affichant la progression des tâches **Celery**).
- La récupération de séries chronologiques brutes pour la visualisation peut entraîner le plantage du navigateur si elles ne sont pas paginées.
- Utiliser l'agrégation côté serveur (agrégats continus **PostgreSQL + TimescaleDB**) avant d'envoyer les données au serveur frontal.
- Si nécessaire, diffuser les données via **WebSockets** pour des mises à jour en temps réel.

Doutes : Backend

- **Django REST Framework** (DRF) : Envisager d'utiliser **Django Ninja** pour des performances API plus rapides.
- Bien utiliser les hypertable avec **TimescaleDB**
- **PostGIS** : L'interrogation simultanée de données spatiales et de séries chronologiques peut être coûteuse, il faut donc optimiser avec l'indexation spatiale (GIST, BRIN, etc.).
- **Celery** : Configurer le suivi des tâches (**Flower** ou **Prometheus**).
- L'optimisation des requêtes est cruciale (utilisez **EXPLAIN ANALYZE** dans **PostgreSQL**).